Using STAR-CCM$^+$ With Virginia Tech's ARC Server

A Collection of Example Scripts

Jeffrey Filer

PHASE Research Group
Virginia Tech

April 1, 2026

---

1 Introduction & Setup

1.1 An ARC Account

The Virginia Tech Advanced Research Computing Department `https://arc.vt.edu/` provides computing resources across the university. They have an excellent setup for enrolled students as the services are freely available (with some caps on maximum nodes and total core-hour usage). ARC provides additional allocations to research groups. With a research group allocation, additional total wall times are permitted (how long a program can run for), core-hours, memory-use, is permitted.

Start by trying to log in to `https://ood.arc.vt.edu/`. Most resources available are found here. If you don't have an account yet, it will prompt you to create one using ColdFront: `https://coldfront.arc.vt.edu/`. This is also where your account is paired with projects to get the additional allocations (and where professors create projects & assign your account).

Once you have logged in to ARC Open OnDemand (OOD), you should see the following page.

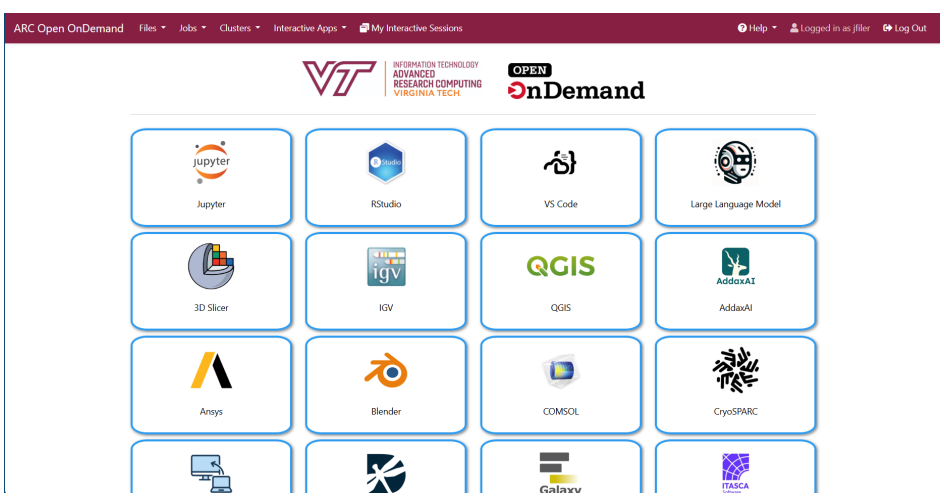

Figure 1: OOD Home Page

Under the "Files" tab are the directories you have access to. With no allocation, you should only see "Home Directory". This is a real directory accessible in the cluster under your username. For me, I have rights over `/home/jfiler`. As you are added to projects, those will also appear under "Files". Note that in the example scripts below, I will commonly reference the base project I work

within (`/projects/ng_inletcfd`) or my own home directory. If you do not change these paths, the script will fail as you likely do not have write access for those directories.

The next tab to make note of is the "Clusters". This is a fast way to access the terminal from any computer with internet access. However, there are benefits to moving to SSH instead using your local PowerShell or Linux command lines. For example, the website will time out, you have to manually upload through the OOD site (instead of using SCP), and it just feels more professional.

ARC has also provided many expansions to their "Interactive Apps". This is a powerful set of tools that allow you to run computationally intense programs with a GUI using the ARC resources. There even is a STAR-CCM$^+$ app. Like a normal job, you request a time and number cores. Once the app starts, it opens as a website. I have spent little time with this functionality so far. Programs typically are multiple versions behind, have difficult to use GUIs (for example you can't Ctrl-C), and need to wait a significant amount of time start). Thus, this document focuses on setting up fully command line jobs. I will note that for US3D using the Paraview app will likely be useful as you don't need to download large datafiles to view them locally and can simple see them through the cluster.

Finally, on the top right of the OOD site is the "Help" tab. Under this, click "ARC Documentation": `https://docs.arc.vt.edu/`. This has an endless amount of info and is a critical resource for diagnosing issues, understanding allocations, and navigating ARC.

1.2 Using SSH

SSH allows you to connect from your local computer to another computer/network. I personally prefer running through Windows PowerShell (as Linux failed when I first started). However, these commands are either the same or have similar analogs in Linux. On your local computer, open PowerShell and type (replacing username with yours):

```
ssh username@tinkercliffs1.arc.vt.edu
```

Listing 1: ARC SSH Command

I have occasionally had connection issues with SSH where it would time out. In this case I found switching to the IPv4 helped by using `ssh -4`.

I will also make note of the SCP and RSYNC commands. SCP is simple and intuitive. RSYNC is better for copying large datasets. Here is an example SCP command I often used to update a CSV file which was holding status information for all my datapoints (optional use of IPv4):

```
scp -4 jfiler@tinkercliffs1.arc.vt.edu:/projects/ng_inletcfd/Fall25Research/sim_data.csv
    "C:\Users\jfiler\Documents\Fall 2025 Simulations\Server Simulations"
```

Listing 2: Example SCP Command

And an example RSYNC command that copied a bunch of data files while not grabbing .sim files (you can remove alot of this but it gives an example of what you can do):

```
rsync -avz --ignore-existing --no-perms --no-owner --no-group --no-times --inplace --
    exclude='*.sim' --exclude='*.sim~' --progress jfiler@tinkercliffs1.arc.vt.edu:/
    projects/ng_inletcfd/Fall25Research/Simulations/ "/mnt/c/Users/jfiler/Documents/Fall
    2025 Simulations/Server Simulations/Simulations 1201"
```

Listing 3: Example SCP Command

1.3 Using Linux Terminal Commands

Knowing how to navigate around a Linux terminal is an important skill for using ARC. This website seems pretty good: `https://codelucky.com/linux-commands-tutorial/`. Here are a couple key commands:

Table 1: Common Linux Commands Used on ARC

Command	**Arguments**	**Description**
`pwd`	–	Prints the current working directory, useful for confirming where you are in the filesystem.
`cd`	*path*	Changes the current directory. Use `cd ..` to move up one directory level and `cd ~` to return to your home directory.
`ls`	`-l`, `-a`	Lists files in the current directory. The `-l` flag shows detailed information, while `-a` includes hidden files.
`man`	*command*	Displays the manual page for a command, including available options and usage examples.
`mkdir`	*directory*	Creates a new directory. Use `mkdir -p` to create nested directories if they do not already exist.
`cp`	*source destination*	Copies files or directories. The `-r` flag is required when copying directories.
`mv`	*source destination*	Moves or renames files and directories. This command is also commonly used for renaming files.
`dos2unix`	*file*	Converts text files created on Windows systems to Unix format, which is required for scripts to run correctly on the cluster.
`clear`	–	Clears the terminal screen for improved readability.

Note that ARC is pretty resilient. You (probably) can't delete anything outside of your own directory. So, the scope of what you can mess up is pretty small (be willing to try things). Be aware of a couple dangerous commands though:

- sudo - unlikely this will often work on ARC. But especially on your own computer, if you see this in a command you are about to do (or copy), this about what it says to do. Used improperly you can easily erase your computer
- rm - this is the remove command. Again -r (recursive) or -f (force) flags can be dangerous. The safest method is -i (interactive)

2 Hello World!

Let us run your first job! As we go along, we will take note of more advanced features as we work towards running a full STAR-CCM$^+$ job. Choose a directory to work with. Make it in the OOD site or using mkdir in the command line (maybe in your home directory). I will be in `/projects/ng_inletcfd/Training`. To practice a series of Linux commands. Try the following sequence (hitting enter for each new line) to create a test text file, print it, and remove it (starting in your directory in the cluster in the terminal).

```
vim test.txt
(Insert Key)
Test
(Escape Key):wq
cat test.txt
rm -i test.txt
y
```

Listing 4: Example Text File

Now, following the same process using vim to create a text file (or by using the OOD "Files" tab), create the following file. Call it `hello_world.txt`. Paste in the following script:

ISSUE: This seems to require an allocation even with personal account

```
#!/bin/bash
# Hello World Script
# sbatch hello_world.txt

#SBATCH -J hello-world --account=ng_inletcfd
#SBATCH --partition=normal_q --nodes=1 --ntasks-per-node=1 --cpus-per-task=1 --time
    =0-00:10:00

echo "Hello world from..."; hostname
echo "Working directory is: "; pwd
echo "Changing directory to $SLURM_SUBMIT_DIR"
cd $SLURM_SUBMIT_DIR
module reset
```

Listing 5: hello_world.txt

Now, in the same directory, run `sbatch hello_world.txt`. To see our job in the queue, run squeue -u username. If you don't see anything, it probably already completed! We told the job to send its output to our working directory so you should see it if your run `ls`. Print it out with cat (now is a good time to know that pressing TAB will fill in the remaining portion of the file name once you start writing it). You should see something like this:

```
[jfiler@tinkercliffs1 Training]$ ls
dmLaunch.txt     rocket_example.sim  rocket_project.dmprj  run_sim_macro.txt slurm
    -4384097.out hello_world.txt rocket_macro.java runDM.txt run_sim.txt
[jfiler@tinkercliffs1 Training]$ cat slurm-4384097.out
Hello world from...
tc004
Working directory is:
/projects/ng_inletcfd/Training
Changing directory to /projects/ng_inletcfd/Training
Resetting modules to system default. Reseting $MODULEPATH back to system default. All
    extra directories will be removed from $MODULEPATH.
```

Listing 6: Hello World Output

If you ever need to stop a job, running `scancel -u username` stops all your current jobs. Lets walk through a couple of those argument flags for SBATCH. ARC provides extensive documentation SLURM interaction here: `https://docs.arc.vt.edu/usage/more-slurm.html`.

Table 2: Common `SBATCH` Argument Flags

Flag	Description
`--account`	Specifies the project or allocation to charge compute time against. This must be set to the appropriate project name; using a personal account is no longer supported.
`--partition`	Selects the queue (partition) on which the job will run. The default partition `normal_q` is suitable for most jobs. Other partitions may be required for high-memory or GPU workloads. See the TinkerCliffs documentation for details: `https://docs.arc.vt.edu/resources/compute/00tinkercliffs.html`.
`--nodes`	Requests the number of compute nodes. A single node provides substantial resources (up to 128 CPU cores), and most jobs should request only one node unless explicitly designed for multi-node parallelism.
`--ntasks-per-node`	Specifies the number of parallel tasks launched on each node. Together with `--cpus-per-task`, this determines how the node's CPU resources are divided among tasks.
`--cpus-per-task`	Defines the number of CPU cores allocated to each task. For example, requesting 8 tasks with 8 CPUs per task uses 64 cores on a node.
`--time`	Sets the maximum wall-clock runtime for the job, formatted as `d-hh:mm:ss`. Jobs exceeding this limit are terminated. Accurate time requests help the scheduler efficiently prioritize jobs.
`--output`	Controls the file path and name for standard output. If not specified, Slurm writes output to a default file in the submission directory.

Keep in mind that the more wall time, nodes, and CPUs you ask for, the longer it will take for your job to start. Unless you are running during big ARC downtime, asking for a full node can take days to start. I have had great success with 1 node, 4 tasks, 4 CPUs, and a 5 hour wall time which will commonly start within minutes. Finally, CFD can get quite memory intensive. The amount of memory available depends on the number of CPUs (unless you set it explicitly). Otherwise, you get my favorite out of memory error:

```
Server process exited with code : 137
slurmstepd: error: Detected 1 oom_kill event in "".batch. Some of the step tasks have
    been OOM Killed
```

Listing 7: Out of Memory):

Finally, note that there are limits to the resources you can get. Details on this for Tinkercliffs can be found here: `https://docs.arc.vt.edu/resources/compute/00tinkercliffs.html`. However, for the standard project allocation, note the following limitations for normal_q:

- Max Wall Time: 7 Days
- Max Number of CPUs: 10,496
- Max Memory: 22,845G

3 Running a Simple STAR-CCM$^+$.sim File

The first step of running CFD on the cluster is build a .sim file locally on your computer. There are three levels to this:

1. I want to build a .sim file and have it run till the max iterations. I'll then download it and look at it locally.
2. I have an automated script which I want to run on a .sim file.
3. I want to run a design manager project on the cluster.

First, we will focus on the first situation in which we run an existing sim file. Note that if you ever need more information about STAR-CCM$^+$ SLURM syntax, you can just run `$STAR_PATH/starccm+`. It will get upset and spit out all the available input arguments. You can also check the SLURM documentation from STAR-CCM$^+$ itself through the "Command-Line Reference" page. Create a new file `run_sim.txt` or copy from the example files. Make sure to correct the file paths for the `--output` argument, `STAR_PATH`, and `SIM_PATH`. Note that if the output file destination fails it should default to your home directory.

```
#!/bin/bash

# Example STARCCM Run File (for rocket_example.sim)
# sbatch run_sim.txt

#SBATCH -J starsim --account=ng_inletcfd
#SBATCH --partition=normal_q --nodes=1 --ntasks-per-node=4 --cpus-per-task=8 --time
    =0-00:30:00
#SBATCH --output="/projects/ng_inletcfd/Training/slurm-starsim-%j.out"

echo "Batch information | Job ID: $SLURM_JOBID | Nodes: $SLURUM_NNODES | Tasks:
    $SLURM_NTASKS | Num Parallel: $SLURM_NPROCS | CPUs/Task: $SLURM_CPUS_PER_TASK"

STAR_PATH=/apps/common/software/STAR-CCM+/18.02.008-R8/STAR-CCM+18.02.008-R8/star/bin/
SIM_PATH=/projects/ng_inletcfd/Training/rocket_example.sim

$STAR_PATH/starccm+ -batch mesh,run $SIM_PATH -batchsystem slurm

module reset

echo "SCRIPT COMPELTE"
```

Listing 8: run_sim.txt

Run this by calling `sbatch run_sim.txt`. It is easiest to open the resulting output file using the OOD site. As it runs, you can refresh the page (Ctrl+R) to refresh the output and watch the iterations. Once completed, the file will save as `rocket_example@0255.sim` and can be downloaded to be viewed locally on your computer.

In the second situation, you can run a Java macro once the simulation is loaded. You can build a Java macro by recording existing operations and assembling them into the script. This was done for `rocket_example.sim` to run for 1,500 iterations with a coarse mesh and then 1,000 iterations with a finer mesh and save on top of `rocket_example.sim`. This macro is included in the example files (`rocket_macro.java`). The save path detailed in the macro must be edited. Note that this simulation will run in serial mode as parallel cores were not explicitly defined in the following call.

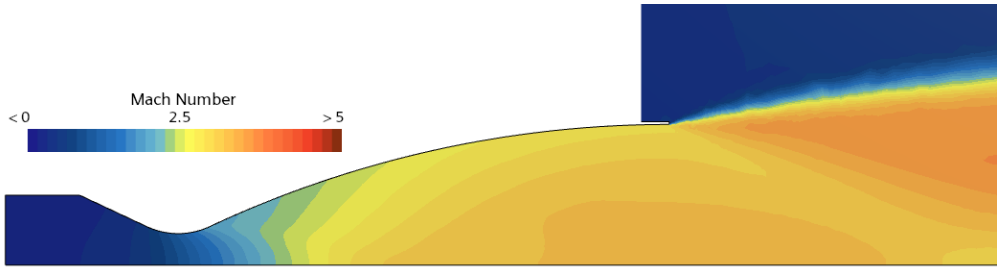

Figure 2: Mach number profile from batch ran simulation.

```
#!/bin/bash

# Example STARCCM Run File (for rocket_example.sim)
# sbatch run_sim_macro.txt

#SBATCH -J starsim --account=ng_inletcfd
#SBATCH --partition=normal_q --nodes=1 --ntasks-per-node=4 --cpus-per-task=8 --time
    =0-00:30:00
#SBATCH --output="/projects/ng_inletcfd/Training/slurm-starsim-%j.out"

echo "Batch information | Job ID: $SLURM_JOBID | Nodes: $SLURUM_NNODES | Tasks:
    $SLURM_NTASKS | Num Parallel: $SLURM_NPROCS | CPUs/Task: $SLURM_CPUS_PER_TASK"

STAR_PATH=/apps/common/software/STAR-CCM+/18.02.008-R8/STAR-CCM+18.02.008-R8/star/bin/
SIM_PATH=/projects/ng_inletcfd/Training/rocket_example.sim
MACRO_PATH=/projects/ng_inletcfd/Training/rocket_macro.java

$STAR_PATH/starccm+ -load $SIM_PATH -batch $MACRO_PATH -batchsystem slurm -np
    $SLURM_NTASKS

module reset

echo "SCRIPT COMPELTE"
```

Listing 9: run_sim_macro.txt

Finally, lets consider the STAR-CCM$^+$ Design Manager. You can turn any .sim file into a Design Manager project (.dmprj file) by right clicking the name at the top left of the GUI and clicking "Create Design Manager Project." The Design Manager is complicated with lots of functionality, so I am not going to attempt addressing its different features (as I barely know a fraction). Instead, I will focus on the changes that must be made to run a .dmprj file on ARC.

There are three methods for running the design manager. Each has its benefits. The first, is "Default Direct" which is the standard setup. You can run the dmRun.txt script below using this method. It will run with create one job where all simulation will run. The second, is "Preallocation." There is a fair amount of documentation for this approach; however, for a significant number of large simulations this is difficult as it requires requesting many nodes and cores at once. On ARC, this is possible, but it may take many days for the job to actually start. The final method, the one I recommend, creates separate jobs for each design in the design manager. It does not appear to have a designated name, so let us call it "True Batch Mode." It keeps job submission low weight so they can start quickly, and they can run completely in parallel with the design manager node

controlling and gathering data. This makes it more complex that the other two modes. Unlike the other cases, two batch scripts must be created. One is similar to the previous scripts with a new flag for the design manager, which starts the Design Manager Project. Then, in the GUI we will assign another batch script which the Design Manager Project program will run to start jobs on the cluster. Let's start with the second script which starts repeated simulations.

1. Under the "Tools" header, right click "Compute Resources" and add a Linux Cluster resource (Note the default being "Default Direct").

2. Edit the Linux Cluster resource. Change the "Job Submit Command" to `sbatch` and the "Script File" `/projects/ng_inletcfd/Training/dmLaunch.txt` (edit to your own path). Make all other fields blank.

3. Under Design Study/Settings/Run Settings change the resource type to "Linux Cluster". Save the file and upload it to ARC. You can also switch back to direct to run locally (or as "Default Direct" on the cluster).

4. Double click the reference sim at the top of the tree. It should open a dialogue showing the reference path to the simulation locally. Make sure this matches for the cluster. Ensure that the path is linux style. The easiest way is to store the .sim file and the .dmprj file in the same folder so there is not extra file path and it works fine for both Windows and Linux.

5. Add the `dmLaunch.txt` file to your directory on the cluster.

```
#!/bin/bash
#SBATCH -J DM_star --account=ng_inletcfd
#SBATCH --partition=normal_q --nodes=1 --ntasks-per-node=4 --cpus-per-task=4 --time
    =0-00:30:00
#SBATCH --output="/projects/ng_inletcfd/log/slurm-DM-%j.out"

STAR_PATH=/apps/common/software/STAR-CCM+/18.02.008-R8/STAR-CCM+18.02.008-R8/star/bin/

STARCCMP=$STAR_PATH/starccm+

echo "Working directory: $PWD"

DMRUNFILE=$(ls *.mdxin 2>/dev/null | head -1)
echo "Found dmrunfile: $DMRUNFILE"

# Read the exact options DM prepared for this design point
EXECOPTS=$(cat dm_design_execute_options.txt)
echo "Execute options: $EXECOPTS"

# Theoretically, -dmnoshare should share the license. Does not seem to do that

$STARCCMP -bs slurm -dmnoshare -np $SLURM_NTASKS -rsh /usr/bin/ssh $EXECOPTS

echo "Job script complete."
```

Listing 10: dmLaunch.txt

Finally, add the following script to your directory to launch the design manager. Then run using `sbatch runDM.txt`

```
#!/bin/bash
# sbatch runDM.txt

#SBATCH -J DM_proj --account=ng_inletcfd
#SBATCH --partition=normal_q --nodes=1 --ntasks-per-node=4 --cpus-per-task=4 --time
    =0-02:00:00
#SBATCH --output="/projects/ng_inletcfd/Training/project-out-%j.out"

echo "Starting dmRun.sh"
echo "Batch information | Job ID: $SLURM_JOBID | Nodes: $SLURUM_NNODES | Tasks:
    $SLURM_NTASKS | Num Parallel: $SLURM_NPROCS | CPUs/Task: $SLURM_CPUS_PER_TASK"

PRJFILE=/projects/ng_inletcfd/Training/rocket_project.dmprj

STAR_PATH=/apps/common/software/STAR-CCM+/18.02.008-R8/STAR-CCM+18.02.008-R8/star/bin/
STARCCMP=$STAR_PATH/starccm+

# Define license settings below
SIMOPTS='-power'
DMPLICO='-dmnoshare'

$STARCCMP -batch run ${DMPLICO} -passtodesign ${SIMOPTS} -dmproject ${PRJFILE}
```

Listing 11: runDM.txt

Once the script has completed, you can download `rocket_project.dmprj` and the associated folder to view the completed cases.